The Role of Swear Words in Toxicity Detection

Eugene Wong
University of Gothenburg
LT2326 Machine Learning for Statistical NLP: Advanced
guswonju@student.gu.se

Abstract

Warning: *this paper contains content that may be offensive or upsetting.*

Does swearing equal toxicity? Simple toxicity detection systems often treat swear words as an immediate indicator of toxic behavior. However, swearing and toxicity do not always align. People swear casually when venting or expressing strong emotions. In contrast, offensive remarks, serious harassment, and threats often use swear-free language. In this paper, we evaluate how the presence or absence of swear words affects toxicity classifiers across a sliced evaluation framework. We find that while a fine-tuned transformer outperforms traditional models by a small margin, it still struggles significantly to recognize non-toxic comments when they contain casual swearing. Our findings show that model architecture cannot overcome training dataset bias, highlighting the need for understanding conversational intent rather than reacting to vocabulary.

1 Introduction

Online forums and social media platforms struggle to create a safe space for users because it is difficult to moderate content accurately. Automated moderation systems often use keyword blocklists as an initial filter, while machine learning classifiers often learn to treat swear words as a shortcut to toxicity (Zhou et al., 2021).

However, in everyday conversations, swearing and toxicity are not the same thing (Pamungkas et al., 2020). People casually swear for humor, excitement, emphasis, or even frustration without meaning any harm:

> *"This article is fucking amazing!"*

In contrast, an abusive comment or threat can be phrased in polite language without any swear words:

> *"You better watch your back when you walk home tonight."*

We can group these failures into two main categories:

1. **False Alarm:** Non-toxic comments containing casual swearing are falsely flagged as toxic.
2. **Blind Spot:** Swear-free toxic comments are missed.

To investigate how much the moderation systems are impacted, we formulate our research question:

> *How does the presence or absence of swear words affect the performance of toxicity detection systems?*

We hypothesize that models make the most mistakes when swearing and toxicity do not align. However, we expect transformers to outperform traditional bag-of-words models because they can understand words in the context of the entire sentence.

2 Methodology

2.1 Dataset

We use the Kaggle Jigsaw Toxic Comment Classification Challenge dataset[1], which consists of human-labeled Wikipedia comments. The dataset is annotated with six labels: *toxic*, *severe_toxic*, *obscene*, *threat*, *insult*, and *identity_hate*.

However, for this research, we are only interested in knowing whether the comment is toxic or not. Therefore, we merge these six labels into a single binary class `is_toxic`. If any of the six labels is positive, we define it as toxic (`is_toxic` = 1), and non-toxic (`is_toxic` = 0) otherwise.

[1] https://www.kaggle.com/c/jigsaw-toxic-comment-classification-challenge

In the test set, a large number of entries were excluded from evaluation and left unlabeled (marked with -1). We remove these entries, reducing the test set from 153,164 to 63,978. We notice that both splits are severely imbalanced, with only about 10% of the comments labeled as toxic (10.17% in the training set and 9.76% in the test set). Because no validation set is provided, we split the training data into an 80% training set (127,656 comments) and a 20% validation set (31,915 comments), stratifying across both `is_toxic` and `has_swear` to preserve class and swear proportions across splits.

2.2 Data Cleanup

We first apply standard text normalization by unescaping HTML entities, removing surrounding quotation marks and whitespace, and collapsing whitespace and linebreaks. Next, we apply three targeted preprocessing steps:

1. **Wikipedia Markup Removal:** We strip the section headers (`==Header==`) and reply indentation colons (`:`) to make the text cleaner to process.

2. **Token Masking:** We replace web URLs with `[URL]` and IP addresses with `[IP]`. Masking these reduces noise.

3. **Collapsing Elongated Words:** There is frequent use of repeated letters for emphasis (e.g., *"sooooo"*). We collapse three or more identical characters down to two (e.g., *"soo"*) to group elongated words together.

2.3 Swear Word Lexicon

To determine whether comments contain swear words, we need a lexicon. For that, we use the dictionary from `noswearing.com`[2].

However, we notice that the list contains some terms that are not offensive by nature. Thus, we apply a whitelist to explicitly remove these words from our swear word lexicon:

1. **Sexual Orientation:** Words such as *"gay"*, *"lesbian"*, and *"queer"* are legitimate identity terms.

2. **Anatomical Terms:** Words such as *"penis"*, *"vagina"*, *"testicle"*, and *"anus"* are medical and biological terms.

[2] `https://www.noswearing.com`

After removing the whitelisted words, our final lexicon contains 341 swear words.

To prevent false substring matches (such as matching *"ass"* inside *"classic"* or *"hell"* inside *"hello"*), we compile the lexicon into a regular expression using word boundaries (`\b`).

We then define a binary class `has_swear`, where `has_swear` = 1 if the comment contains at least one swear word from the lexicon, and `has_swear` = 0 otherwise.

2.4 Evaluation Framework

Because the dataset is severely imbalanced, standard aggregate metrics cannot tell the full story (Röttger et al., 2021). To isolate the effect of swear words and test our hypothesis, we slice the test set into a 2×2 matrix based on toxicity (`is_toxic`) and whether the comment contains swear words (`has_swear`):

Table 1: Test set comment distribution across toxicity and swear word presence ($N = 63,978$).

	With Swear	No Swear	Total
Toxic	3,234 (5.05%)	3,009 (4.70%)	6,243 (9.76%)
Non-Toxic	2,874 (4.49%)	54,861 (85.75%)	57,735 (90.24%)
Total	6,108 (9.55%)	57,870 (90.45%)	63,978 (100%)

Table 1 reveals two critical challenges:

- **False Alarm Risk:** Among the 6,108 comments containing swear words, toxic comments (3,234, or 52.95%) and non-toxic comments (2,874, or 47.05%) are almost evenly split. Determining the comment's toxicity is essentially doing a coin flip.

- **Blind Spot Risk:** Nearly half of all toxic comments (48.20%, or 3,009 out of 6,243) contain no swear words at all.

In contrast to the balanced swear-word distribution in the test set, the training set exhibits severe distribution skew. 78.22% of all swearing comments in the training set are toxic (6,865 toxic vs. 1,912 non-toxic). This imbalance creates bias that swearing means toxicity.

To quantify performance, we focus on two key metrics:

$$\Delta\text{FPR} = \text{FPR}_{\text{with swear}} - \text{FPR}_{\text{no swear}} \quad (1)$$

ΔFPR measures the **False Alarm Surge**: how much does casual swearing inflate false alarms?

Ideally, a fair model should achieve $\Delta\text{FPR} = 0$, indicating that casual swearing never triggers a false accusation.

$$\Delta\text{Recall} = \text{Recall}_{\text{with swear}} - \text{Recall}_{\text{no swear}} \quad (2)$$

ΔRecall measures the **Sensitivity Drop**: how much does detection drop when a comment contains no swear words? Ideally, a robust model should achieve $\Delta\text{Recall} = 0$, indicating that it catches toxic comments without depending on swear words.

2.5 Models Evaluated

We evaluate five models spanning from simple heuristics to a pre-trained transformer:

1. **Majority Baseline:** Always predicts the majority class (non-toxic). It serves as an accuracy benchmark (90.24%) to demonstrate why accuracy is a flawed metric for an imbalanced dataset.

2. **Random Baseline:** Generates predictions randomly according to the class distribution (10% toxic, 90% non-toxic).

3. **Rule-Based Lexicon Baseline:** Predicts that a comment is toxic if it contains at least one word from our swear lexicon (`has_swear = 1`). This acts as a benchmark for lexical bias.

4. **TF-IDF + Logistic Regression:** Text is vectorized into the top 25,000 unigrams and bigrams using sublinear term frequency scaling, followed by a logistic regression classifier with balanced class weights to compensate for class imbalance.

5. **DeBERTa-v3-base:** A pre-trained transformer that uses disentangled self-attention to process words in relation to their surrounding context. Analyzing token length across 10,000 sampled comments showed that 82.75% fit within 128 tokens, so we set the maximum sequence length to 128. We fine-tune the model for 3 epochs with an effective batch size of 32 (batch size 16 with gradient accumulation), a learning rate of 2×10^{-5}, and mixed precision (`bf16`), selecting the final model from the checkpoint with the highest F_1.

3 Results

3.1 Overall Metrics

Table 2 presents the overall evaluation metrics across all five models.

The Majority Baseline achieves a high accuracy of 90.24% while its F_1 is 0.00%. This proves that standard accuracy is misleading on an imbalanced dataset. A similar pattern is shown by the Random Baseline ($F_1 = 10.16\%$).

The Rule-Based baseline's F_1 of 52.37% (recall of 51.80% and precision of 52.95%) directly reflects the coin-flip analogy we described in Section 2.4.

As expected, DeBERTa achieves the highest overall performance with an F_1 score of 65.83% (an absolute gain of 7.90 percentage points over TF-IDF + Logistic Regression) and a PR-AUC of 80.74% (substantially outperforming TF-IDF's 74.74%).

3.2 Sliced Evaluation and Lexical Bias

Table 3 evaluates our five models across the four slices of our 2×2 matrix.

The Rule-Based baseline perfectly portrays the failure of using keywords to determine toxicity. It achieves 100% recall on toxic comments with swear words, but 0% recall on toxic comments without swear words (a 100.00 percentage point sensitivity drop, ΔRecall). This means it completely fails to identify toxic comments when there are no swear words. In addition, it falsely flags 100% of casual swearing comments (100.00% FPR).

Comparing the TF-IDF baseline against DeBERTa highlights both the strength and the limits of contextual modeling:

- **Where DeBERTa improves (The Blind Spot):** The TF-IDF baseline achieves only 84.65% recall on swear-free toxicity (a 10.72 percentage point sensitivity drop, ΔRecall). DeBERTa uses surrounding context to catch more subtle toxicity, lifting recall to 90.06% and shrinking this gap to 7.00 percentage points (ΔRecall).

- **Where both models struggle (False Alarms):** TF-IDF wrongly flags 65.00% of non-toxic comments containing casual swearing (a false alarm surge of 54.61 percentage points over its swear-free rate, ΔFPR). While DeBERTa's self-attention reduces this false alarm rate

Table 2: Overall test set performance across baselines and DeBERTa.

Model	Accuracy	Precision	Recall	F_1	PR-AUC
Majority Baseline	0.9024	0.0000	0.0000	0.0000	0.0976
Random Baseline	0.8219	0.1000	0.1032	0.1016	0.0978
Rule-Based Baseline	**0.9080**	**0.5295**	0.5180	0.5237	N/A
TF-IDF + Logistic Regression	0.8722	0.4267	0.9020	0.5793	0.7474
DeBERTa	0.9051	0.5074	**0.9369**	**0.6583**	**0.8074**

Table 3: Sliced evaluation across toxic comments (Recall) and non-toxic comments (False Alarm Rate).

	Toxic Comments (Recall)			**Non-Toxic Comments (False Alarm Rate)**		
Model	With Swear	No Swear	ΔRecall	With Swear	No Swear	ΔFPR
Majority Baseline	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Random Baseline	0.1030	0.1034	-0.0004	0.1023	0.1003	0.0020
Rule-Based Baseline	1.0000	0.0000	1.0000	1.0000	0.0000	1.0000
TF-IDF + Logistic Regression	0.9536	0.8465	0.1072	0.6500	0.1039	0.5461
DeBERTa	**0.9706**	**0.9006**	**0.0700**	**0.5828**	**0.0730**	**0.5098**

to 58.28%, it still exhibits a surge of 50.98 percentage points over swear-free comments (ΔFPR).

This confirms our hypothesis that while contextual attention helps improve the performance of toxicity detection, it still cannot overcome the strong correlation between toxicity and swear words learned from a biased dataset.

4 Error Analysis

To understand why the traditional bag-of-words model fails and where DeBERTa shines, we inspect the learned feature weights and analyze error cases from the test predictions.

4.1 The Bag-of-Words Trap: Feature Weights

Table 4: Top learned Logistic Regression coefficients from the TF-IDF baseline.

Top Toxic Features		**Top Non-Toxic Features**	
Feature	Weight	Feature	Weight
`fuck`	$+22.70$	`thanks`	-5.88
`fucking`	$+19.75$	`talk`	-5.01
`stupid`	$+18.67$	`thank`	-4.81
`shit`	$+18.03$	`thank you`	-4.79
`idiot`	$+15.86$	`please`	-4.74

Inspecting the top learned coefficients from the TF-IDF with Logistic Regression model (Table 4) reveals that swear words have massive positive weights, while the opposite, polite expressions, carry much smaller negative weights. As a result, a single swear word can overpower the rest of the sentence and push the prediction toward the toxic class regardless of the surrounding context.

4.2 Where Context Helps: DeBERTa's Rescues

Out of the 2,874 non-toxic comments containing casual swearing, DeBERTa successfully corrects 466 comments that the TF-IDF baseline falsely flags. A clear example is shown below:

> *"Well, a station that loses out on the CW... they may be shit-out-of-luck, but they won't close, as too much money is at stake..."*
>
> **Human:** Non-toxic
> **Has Swear:** Yes
> **TF-IDF:** Toxic ($P(\text{toxic}) = 50.10\%$)
> **DeBERTa:** Non-toxic ($P(\text{toxic}) = 3.76\%$)

TF-IDF flags the comment as toxic because of the isolated word *"shit"*, while DeBERTa recognizes the harmless idiom *"shit-out-of-luck"* in context, predicting with high confidence that it is non-toxic.

Similarly, out of the 3,009 swear-free toxic comments, DeBERTa catches 320 comments that the TF-IDF baseline misses entirely:

> *"he was the gayest person alive...."*
>
> **Human:** Toxic
> **Has Swear:** No
> **TF-IDF:** Non-toxic ($P(\text{toxic}) = 34.23\%$)
> **DeBERTa:** Toxic ($P(\text{toxic}) = 97.07\%$)

This comment does not contain any swear words, so TF-IDF predicts it as non-toxic. DeBERTa, however, reads the sentence as a whole and recognizes the targeted insult, correctly classifying it as toxic.

4.3 Shared Failures

Despite DeBERTa's contextual capability, there are still cases where it fails alongside the TF-IDF model. Across the test set, both models falsely flag 1,402 non-toxic comments containing swear words (48.78% of that slice), and both fail to detect 142 swear-free toxic comments (4.72% of that slice):

1. **Emotional Venting (Shared False Alarm):**
   "Well honestly I cant be fucking bothered tonight. So sleep wee all u editors ans admits . I'm off to bed. CU anaon"
   Human: Non-toxic
   Has Swear: Yes
   TF-IDF: Toxic ($P(\text{toxic}) = 99.43\%$)
   DeBERTa: Toxic ($P(\text{toxic}) = 99.38\%$)
   In this comment, the swear word is used casually to express an emotion. However, both models predict with over 99% certainty that the comment is toxic.
2. **Polite Threats (Shared Blind Spot):**
   "Nice job inserting your left-wing garbage in Jan Brewers article. I removed it and if you vandalize the article again you will be blocked from editing on Wikipedia."
   Human: Toxic
   Has Swear: No
   TF-IDF: Non-toxic ($P(\text{toxic}) = 47.34\%$)
   DeBERTa: Non-toxic ($P(\text{toxic}) = 9.74\%$)
   This hostile comment contains no swear words and adopts an administrative tone, successfully deceiving both models. DeBERTa assigns only a 9.74% toxicity probability, failing to detect the underlying coercive threat.

5 Conclusion

5.1 Takeaways

In this paper, we evaluated how the presence or absence of swear words affects the performance of toxicity detection systems using a 2×2 sliced evaluation framework on 63,978 human-labeled Wikipedia comments. We establish that swearing $\neq$ toxicity.

Traditional bag-of-words models suffer from severe lexical bias. The presence of swear words causes the false alarm rate to surge from 10.39% to 65.00% (a surge of 54.61 percentage points, ΔFPR). It also causes a 10.72 percentage point drop in detection recall (ΔRecall) when toxic comments are swear-free.

DeBERTa, despite its contextual self-attention, still falsely flags **58.28%** of non-toxic comments containing casual swearing. This shows that even a contextual transformer cannot overcome lexical bias when the training distribution is heavily skewed. Because 78.22% of comments containing swear words in the training set are labeled as toxic, DeBERTa learns to rely on swear words as a shortcut. Effective moderation requires understanding conversational intent, not reacting to vocabulary.

5.2 Limitations

We acknowledge several limitations in our experimental setup:

1. **Lexicon Coverage and Obfuscation:** Our swear word lexicon relies on an unofficial dictionary of 341 words and cannot be used as a gold standard. In addition, keyword matching cannot detect obfuscation and leetspeak (e.g., *"f*ck"* or *"sh1t"*). This means that some comments are still falsely classified as swear-free.
2. **Domain Specificity:** The dataset comes from Wikipedia comments, which exhibit different conversational norms, wording, and emotional tone compared to other domains like social media platforms or forums.
3. **Binary Target Consolidation:** Our experiment is simplified by merging the six labels into a single `is_toxic` class. However, re-evaluating the whole experiment under the subcategories of toxicity could potentially yield new insights.

5.3 Future Work

In order to improve the performance of toxicity detection, we should focus on improving dataset balance and annotation quality rather than solely increasing model size.

1. **Counterfactual Data Augmentation:** Training datasets could be augmented by inserting casual swear words into non-toxic comments without changing the intent and toxicity (e.g., *"This is awesome"* $\rightarrow$ *"This is fucking awesome"*). This would teach the models that the presence of swear words does not imply toxicity.
2. **Profanity-Aware Tokenization:** Standard tokenizers struggle with obfuscated swearing (e.g., *"f*ck"* or *"sh!t"*). We could explore training specialized domain tokenizers to better recognize obfuscated swear word variants as unified tokens.
3. **Fairness-Regularized Objectives:** We need to adjust the loss functions so that penalties account for class and slice imbalances.

References

Endang Wahyu Pamungkas, Valerio Basile, and Viviana Patti. 2020. Do you really want to hurt me? predicting abusive swearing in social media. In *Proceedings of the Twelfth Language Resources and Evaluation Conference*, pages 6237–6246, Marseille, France. European Language Resources Association.

Paul Röttger, Bertie Vidgen, Dong Nguyen, Zeerak Waseem, Helen Margetts, and Janet Pierrehumbert. 2021. HateCheck: Functional tests for hate speech detection models. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 41–58, Online. Association for Computational Linguistics.

Xuhui Zhou, Maarten Sap, Swabha Swayamdipta, Yejin Choi, and Noah Smith. 2021. Challenges in automated debiasing for toxic language detection. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pages 3143–3155, Online. Association for Computational Linguistics.